Banking System

Sprint 3: System Design

Table of Contents

Table of Contents **2**
System Interaction with the Environment **3**
Dependencies 3
Assumptions 3
CRC Cards **3**
Frontend **6**
Overview 6
Navigation 6
Components 6
Sign-up 6
Dashboard 7
Deposit & Withdraw 7
E-Transfer 7
Login 7
Loan-Management 7
Backend **7**
Overview 7
DTOs 7
Schema 7
User 7
Transaction 8
API Calls 8
/addUser (POST) 8
/account/balance/{accountNumber} (GET) 8
/login (POST) 8
/deposit (POST) 8
/withdraw (POST) 9
/transfer (POST) 9
/transactionHistory/{accountNumber} (GET) 9
/recentTransaction/{accountNumber} (GET) 9
Installation **9**

System Interaction with the Environment

Dependencies

1. NodeJS is used to handle server-side programming and dependency importing.

2. Angular framework is used to generate the frontend view for customers.
3. Java (Spring) is used for backend logic to manage the database.
4. MySQL database is used to model and store data.

Assumptions

1. The system will be deployed and accessed over a modern web browser.
2. An internet connection is necessary for functionality of the system.

CRC Cards

Class:Account	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes and methods of accounts ○ ID, balance ● Getters and setters	Collaborators: ● Transaction ● Users

Class:Transaction	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes of a transaction ○ ID, Account, amount, date ● Getters and Setters	Collaborators: ● Account

Class:Users
Parent Class: N/A Subclasses: N/A

Responsibilities: ● Models the attributes of a user ○ ID, name, amount, email, password, balance ● Getters and Setters	Collaborators: ● Account ● UsersController ● BalanceRequest ● LoginRequest ● SignupRequest ● TransactionRequest ● TransferRequest

Class:UsersController	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Provides API calls ○ Deposit ○ Withdraw ○ Transfer ○ TransactionHistory ○ RecentTransaction ● Getters and Setters	Collaborators: ● Users ● UserService

Class:BalanceRequest	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes of a balance request ○ Account ○ Amount ● Getters and Setters	Collaborators: ● Users ● UserService

Class:LoginRequest	
Parent Class: N/A Subclasses: N/A	
Responsibilities:	Collaborators:

• Models the attributes of a login request ○ ID ○ Password • Getters and Setters	• Users • UserService

Class:SignupRequest	
Parent Class: N/A Subclasses: N/A	
Responsibilities: • Models the attributes of a signup request ○ Name ○ Email ○ Password ○ ExitingEmail • Getters and Setters	Collaborators: • Users • UserService

Class:TransactionRequest	
Parent Class: N/A Subclasses: N/A	
Responsibilities: • Models the attributes of a transaction request ○ ID ○ Amount • Getters and Setters	Collaborators: • Users • UserService

Class:TransferRequest	
Parent Class: N/A Subclasses: N/A	
Responsibilities: • Models the attributes of a transfer request ○ FromEmail ○ ToEmail	Collaborators: • Users • UserService

○ Amount ● Getters and Setters	

Class:UserService	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Handles functionality for UserController	Collaborators: ● Users ● UserController ● BalanceRequest ● LoginRequest ● SignupRequest ● TransactionRequest ● TransferRequest

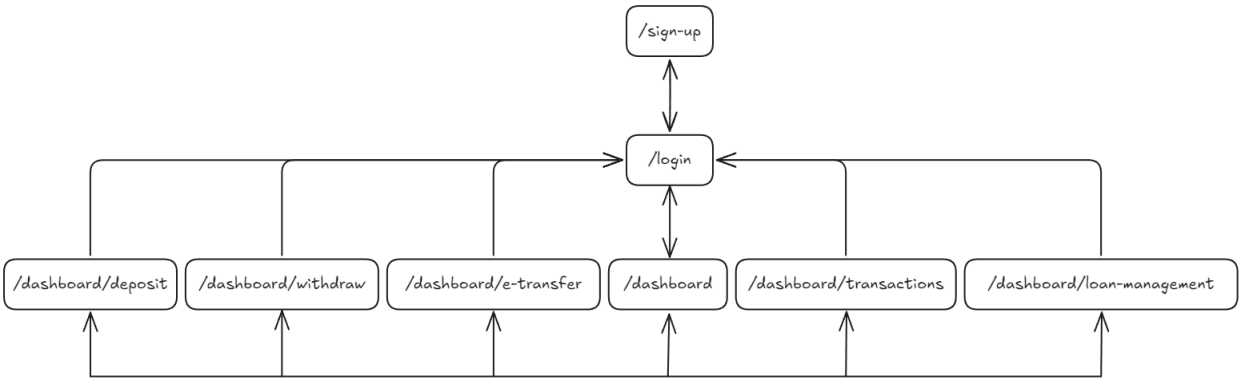
Frontend

Overview

The frontend can be broken down into the various menus the user can access. With exception to the initial login page, every component can be interacted with in any order.

Navigation

1. Users initially enter the banking system through the signup page.
2. From there, they either create an account or login with existing credentials on the login page.
3. Finally, the user can freely and exit all remaining dashboard pages depending on their needs.



Components

Sign-up

- Requests account creation from the user

Dashboard

- Fetches an account's balance and recent transactions

Deposit & Withdraw

- Prompts the user to input an amount of money to the owner's account

E-Transfer

- Prompts the user to input an amount of money to send to another user's account

Login

- Requested the user's email and password
- Allows access if the credentials are found in the database

Loan-Management

- Displays ongoing loan balances as well interest rates

Backend

Overview

The backend is modelled using java, represented in a SQL format, handled with Spring Boot and hosted online via AlwaysData.com.

DTOs

The java controller for accounts and customers uses an intermediate format that stores data with public getters and setters to update the model.

Schema

User

```
CREATE TABLE users(
  email          VARCHAR(255) NOT NULL PRIMARY KEY,
  account_number VARCHAR(255) NOT NULL UNIQUE,
  balance        DOUBLE       NOT NULL,
  name           VARCHAR(255),
  password       VARCHAR(255) NOT NULL
);
```

Transaction

```
CREATE TABLE transaction(
  id             BIGINT(20)   NOT NULL PRIMARY KEY,
  amount         DOUBLE,
  description    VARCHAR(255),
  timestamp      DATE,
  type           VARCHAR(255),
  account_number VARCHAR(255) NOT NULL,
  FOREIGN KEY (account_number) references users(account_number)
);
```

API Calls

/addUser (POST)

Given request body of:

- name
- email
- password

Create an account with those parameters, with a randomized account number, and balance of 0.00.

/account/balance/{accountNumber} (GET)

With the given "accountNumber", it returns the balance and the associated account number.

/login (POST)

Given request body of:

- email
- password

Check whether the user is valid. If it is, return an OK response. Otherwise, throw a BAD_REQUEST or ERROR response.

/deposit (POST)

Given request body of:

- accountNumber
- amount

Deposit money equal to amount into the specified account.

/withdraw (POST)

Given request body of:

- accountNumber
- amount

Withdraw money equal to the amount into the specified account.

/transfer (POST)

Given request body of:

- fromEmail
- toEmail
- amount

Transfer funds equal to the amount from the account associated with "fromEmail" to the account associated with "toEmail".

/transactionHistory/{accountNumber} (GET)

Given "accountNumber", get a list of all transactions associated with that account.

/recentTransaction/{accountNumber} (GET)

Given "accountNumber", get the most recent transaction associated with that account.

Installation

To develop on this project, you currently need the following:

- NodeJS (installation of other dependencies)
- Angular CLI
- Java (1.8)
- IntelliJ
- VSCode

First, clone the repository.

To run the backend:

1. Open IntelliJ with the path folder backend/bankApp
2. To run the backend, run the BankAppApplication.java with Java 23

To run the frontend:

1. Open VScode with path folder frontend
2. Run *npm install* in the terminal
3. Run *ng serve* in the terminal